

QRX 0.0.9 A-Z Handbook

QRX 0.0.9 — A-Z Handbook

Canonical offline handbook for QRX Core, GUI Wallet, QRX Drive, QRX-Net and AURA

Documentation phase: **0.0.9.49 + Genesis Freeze extensions through Apps UX/i18n / Phase 190**

Branch state covered: **0.0.9.0 through the current 0.0.9 Genesis Freeze source snapshot**

Current Genesis Freeze regression baseline: **123/123 registered Core tests PASS**, including on-chain protocol governance, protocol readiness, QRX Upscaler capability profiles, the `.qrxapp` application foundation and the Phase 190 multilingual/full-width Apps UX gate.

This handbook explains the intended QRX user and operator workflows from first installation to advanced provider operation. QRX remains software that should be treated conservatively: keep backups, verify release hashes, test recovery before relying on funds, and do not treat a passing test suite as a substitute for an independent security audit.

1. QRX in one page

QRX is a native blockchain/network stack centered on **QUB (QUBITCOIN)** and extended by several layers that share the same wallet identity and node infrastructure:

- **QRX Chain / QUB** — balances, transactions, staking, delegation, validators, finality, governance and native assets.
- **VELOCITY** — high-throughput transaction execution, MVCC/speculative execution, native markets and cross-chain settlement foundations.

- **QRX Drive** — decentralized encrypted storage with erasure coding, provider discovery, multi-provider downloads, resumability and repair.
- **QRX-Net** — `.qrx` names, decentralized website packages, browser resolution, DApp permissions, advertising/reward policy and youth-safety controls.
- **AURA / Proof of Useful Compute** — distributed useful compute, model/expert distribution, MoE scheduling, verified runtimes, provider discovery and task routing.
- **BTC Light / Bitcoin SPV** — local Bitcoin key/service integration and SPV verification used by BTC-related wallet functions and Quantum Swaps.
- **Quantum Swaps / Velocity Cross-Chain** — HTLC and Bitcoin-SPV-backed cross-chain settlement flows.
- **Markets / Agents & Kraken** — QRX-native order book plus explicitly authorized agent/gateway paths for external execution.
- **Generals** — a separate QRX-native game/application layer that uses QRX wallet/network primitives.
- **QRX App Foundation** — local third-party app packaging (`.qrxapp`), App Registry, sandboxed App Host, permission-gated Mini JS SDK and Developer Mode. Public/decentralized distribution is intentionally a 0.0.10 layer.
- **QRX Upscaler** — local image/batch/video upscaling foundation with capability-aware Apple Silicon, Raspberry Pi 5 and ODROID-N2/N2+ handling; distributed frame processing remains gated behind Proof of Useful Compute.

What is consensus-critical and what is not?

Do not assume everything visible in the wallet lives on-chain.

Consensus/state examples: QUB balances, signed transactions, validator/delegation state, native markets, governance state, asset state, PoUC settlement state, domain commitments and other explicitly committed protocol objects.

Dynamic/off-chain examples: current AURA provider load, model chunk availability, WAN latency, relay health, model cache contents, local GUI preferences, local BTC service state and most short-lived discovery data. These can be signed and verified without being written into every block.

Networks

QRX supports the profiles:

- mainnet
- alpha
- testnet
- regtest

The current source schedules **Mainnet genesis for 15 September 2026 at 16:00 UTC / 18:00 CEST**. Nodes can be prepared before that time, but Mainnet block production is prevented before the configured activation timestamp.

Genesis tokenomics and service economics

The Genesis policy deliberately separates **protocol emission** from **paid network services**. Storage, useful compute and advertising do not create a second inflation stream. Their rewards come from QUB already escrowed by the user/advertiser.

Protocol / staking emission

- Genesis block reward: **0.25 QUB every 10 seconds** (25,000,000 atoms).
- Initial gross subsidy: **788,400 QUB/year** at the 10-second target before halvings.
- Halving interval: **12,614,400 blocks** (about four years at target block time).
- Maximum supply remains **21,000,000 QUB**. This is a hard ceiling, not the issuance target of the current schedule. With a 0.25-QUB initial reward and four-year halvings, the idealized subsidy series totals about **6,307,200 QUB** before integer-atom rounding; reaching a Bitcoin-like ~21M emission would require a different pre-Genesis emission design.
- The development-fund share is carved out of the block subsidy (20% year 1, 10% year 2, 5% year 3, 2% thereafter); it does **not** mint on top of the block reward.

Proof of Storage / QRX Drive

A storage contract is funded by the client and split deterministically:

- **97.5%** provider budget
- **2.0%** resilience/repair reserve

- **0.5%** QRX development share

Unused provider/resilience escrow follows the contract refund rules. Storage proof payouts and repair payouts come from those funded pools rather than new QUB issuance.

Proof of Useful Compute / AURA

Normal verified compute is paid from the job owner's authorized compute escrow. Optional FastTrack is capped at **25% of the maximum compute budget** and its surcharge is split:

- **90.0%** to the provider performing the priority work
- **9.5%** to the QRX network/fee pool
- **0.5%** to the QRX development share

Verifier/challenge rewards are also explicitly user-funded in the job's verification budget. Compute cannot mint additional QUB simply because more machines join the network.

QRX-Net advertising

Each verified ad impression is paid from the advertiser's campaign escrow and split:

- **55.0%** delivery providers
- **25.0%** publisher
- **15.0%** viewer reward
- **4.5%** protocol/network pool
- **0.5%** QRX development share

This means greater QRX usage can raise provider/validator/user income through actual demand without changing the monetary emission curve.

2. Installation and first start

2.1 Recommended path for beginners

For a normal desktop user:

1. Install the **QRX GUI Wallet** build matching the operating system and CPU.
2. Start the wallet.
3. Choose the intended network in the lower-left network selector.
4. Create or import a wallet.
5. Make and verify a recovery backup before receiving significant funds.
6. Allow the wallet to start/connect the local QRX node.
7. Use **Dashboard** → **Mainnet Health** (or the equivalent selected network) to confirm sync and peers.

You normally do **not** need to install a separate model runtime, edit JSON, choose GGUF/MLX/CUDA, open a provider port, or manually select AURA experts.

2.2 Supported release targets

The unified builder currently knows these native release targets:

Target	Typical output
Linux x64	Core/CLI + DEB/AppImage wallet
Linux ARM64	Core/CLI + DEB/AppImage wallet
macOS Intel	Core/CLI + APP/DMG wallet
macOS Apple Silicon	Core/CLI + APP/DMG wallet
Windows x64	Core/CLI + MSI/NSIS wallet

Use native runners for final OS packages. The build system intentionally rejects pretending an x86 build is ARM simply by renaming it.

2.3 Building from source

Core-only development build:

```
cd qrx-core  
cmake -S . -B build  
cmake --build build -j
```

Run the native regression suite:

```
cmake -S . -B build-tests -DQRX_BUILD_TESTS=ON  
cmake --build build-tests -j  
ctest --test-dir build-tests --output-on-failure
```

Unified host build:

```
bash scripts/build-all-targets.sh --target host
```

Inspect all build plans without compiling:

```
bash scripts/build-all-targets.sh --all --plan
```

The unified release builder compiles Core/CLI first, then the BTC wallet service and target sidecars, then the Tauri GUI, and finally packages checksummed artifacts.

2.4 Data layout

The shared wallet design uses the user QRX root:

```
~/.qrx/
```

Network-specific chain/node state lives below the selected network, while the shared hybrid wallet identity lives in the common wallet store. The GUI and native Core intentionally point at the same shared root so a wallet does not have to be copied between CLI and GUI.

Never manually merge two wallet directories containing different private-key sets.

2.5 Core/daemon/CLI entry points

The native supported command-line programs are:

- `qrx` — backend/core utility entry point
- `qrx-d` — daemon/node frontend
- `qrx-cli` — local node/wallet control frontend

Simple alpha example:

```
export QRX_PASSPHRASE='your-passphrase'  
./build/qrx-d --network alpha --datadir ./data --wallet node1 --  
listen 127.0.0.1:26661
```

Another terminal:

```
./build/qrx-cli --network alpha --datadir ./data --wallet node1  
getinfo  
./build/qrx-cli --network alpha --datadir ./data --wallet node1  
getwalletinfo  
./build/qrx-cli --network alpha --datadir ./data --wallet node1  
getnewaddress
```

3. Wallet setup, identity, unlocking and recovery

3.1 The QRX wallet identity

A QRX wallet uses a **hybrid signing identity**. The current wallet layout includes Ed25519 and ML-DSA-65 key material. The GUI is designed so normal users deal with the wallet name, address and passphrase rather than individual key files.

Golden rule

The passphrase is not the recovery backup. Keep a recovery pair/back-up independently of the passphrase.

3.2 New wallet checklist

After creating a wallet:

1. Confirm the wallet address is displayed.
2. Set a strong wallet passphrase.
3. Open **Safety Center**.
4. Generate/export an encrypted wallet backup and/or the supported recovery pair.
5. Verify the backup health.
6. Store at least one copy offline.
7. Only then begin using rotating receive identities or holding meaningful QUB.

3.3 Unlocking

The sidebar contains the session unlock control. Enter the wallet passphrase and press **Enter** or **Unlock**. The passphrase is intended to remain in application memory for the session rather than being written to settings.

Use **Lock wallet** when you want the GUI to forget the in-memory passphrase.

3.4 Recovery Center

The recovery flow is create-only: it should restore into a **new wallet name** instead of overwriting an existing wallet directory.

Typical safe recovery sequence:

1. Keep the original wallet folder untouched.
2. Enter a new wallet name.
3. Enter the recovery phrase exactly.
4. Select the matching `recovery.qrxseed` file when requested.
5. Protect the restored keys with a new passphrase.
6. Compare the restored public address/identity with the expected wallet.
7. Test a harmless read operation before considering the recovery complete.

Legacy 0.0.6 recovery pairs can remain valid if retained. Creating a fresh recovery pair for the same keys does not itself change the wallet address.

3.5 Wallet migration

The Wallets view can identify older GUI/legacy locations and copy them into the shared `~/ .qrx` store. The intended migration model is **copy first, verify second, remove old data only later**.

Do not overwrite a functioning source wallet during migration.

4. GUI orientation

The left sidebar contains the current wallet, QUB balance, lock state and every primary application view. The footer contains node status, selected network, RPC/control endpoint and data directory.

The current GUI navigation items are documented one by one below.

5. Dashboard

Purpose: everyday node/wallet status.

The Dashboard shows balance, staked amount, validator information and peers. The **Mainnet Health** card exposes local chain height, how far the node is behind, connection count, supply, best block hash, state root, protocol and network.

Peer Viewer

The optional Peer Viewer reads peer information from the local node. Treat peer addresses/IPs as local operational information; the view is not intended to publish them.

Reward notifications

Validator, delegation and trading/settlement notifications can be toggled independently. Use a minimum-QUB threshold if many tiny rewards would otherwise create noise.

Beginner check

Before sending funds or enabling provider functions, confirm:

- daemon status is online,
 - selected network is correct,
 - height is progressing,
 - peers are connected,
 - wallet is the expected wallet.
-

6. Apps

Purpose: application launcher and local execution boundary for QRX-integrated applications.

QRX 0.0.9 has two kinds of applications in the Apps area:

1. **built-in QRX applications** such as Generals, Browser and QRX Upscaler; and
2. **third-party/local applications** installed through the QRX App Foundation.

Third-party apps are separate trust domains even though they are displayed inside the wallet. They do not become trusted wallet code merely because the user installed them.

6.1 QRX App Foundation in 0.0.9

The Genesis branch deliberately ships only the **local foundation** required for third-party apps:

- `.qrxapp` package format v1;
- Wallet **App Registry**;
- sandboxed QRX **App Host**;
- explicit permission allowlist with per-call enforcement in the Rust/Tauri backend;
- **QRX Mini JS SDK v1**;
- local install/uninstall of `.qrxapp` packages;
- **Developer Mode** for live unpacked app folders;
- the `Hello QRX` reference/demo app;
- no private-key exposure and no app-side transaction signing.

Developer package signatures, QRX Drive publication, QRX-Net discovery, the QRX App Directory, update channels and application compute capabilities are deferred to **0.0.10**. A sideloaded 0.0.9 package is therefore shown as **unverified**; a file named `SIGNATURE` inside the archive does not create trust.

6.2 .qrxapp package format v1

A `.qrxapp` is a bounded ZIP container with a `qrx-app.json` manifest at the package root. A minimal package looks like:

```
hello-qrx.qrxapp
├─ qrx-app.json
├─ index.html
├─ app.js
├─ app.css
└─ icon.png          # optional/reserved for UI use
```

Required manifest fields are `format=1`, `id`, `name`, `version`, `author`, `entry` and `sdk=1`. Permissions must come from the v1 allowlist.

Example:

```
{
  "format": 1,
  "id": "org.example.hello",
  "name": "Hello QRX",
  "version": "1.0.0",
  "author": "Example Developer",
  "entry": "index.html",
  "script": "app.js",
  "style": "app.css",
  "sdk": 1,
  "permissions": [
    "wallet.identity.read",
    "wallet.balance.read",
    "wallet.payment.request",
    "chain.read",
    "network.status.read",
    "app.storage"
  ]
}
```

The installer rejects path traversal (`../`), absolute paths, symlink escapes, unknown permissions, malformed app IDs, unsupported SDK versions and missing entry files. The 0.0.9 package limits are deliberately small enough for a wallet-hosted application: **64 MiB package size, 64 MiB total unpacked size, 16 MiB per file and 256 files.**

6.3 Sandbox and trust boundary

Third-party app code runs in an iframe with:

```
sandbox="allow-scripts"
```

and deliberately **without** `allow-same-origin`. The app does not receive direct Tauri IPC, private wallet material, camera, microphone, geolocation, unrestricted popups/forms or top-level navigation.

The communication path is:

```
Third-party app
  |
  | postMessage
  v
QRX App Host
  |
  | permission check
  v
Rust/Tauri bridge
  |
  v
Wallet / local node read APIs
```

The frontend permission check is not the security boundary by itself: the Rust host checks the permission again for every SDK operation.

6.4 Mini JS SDK v1

The SDK is injected by the QRX App Host; an app does not bundle its own privileged wallet bridge.

Typical calls:

```
const identity = await QRX.wallet.getIdentity();
const balance = await QRX.wallet.getBalance();
const height = await QRX.chain.getHeight();
const network = await QRX.network.getStatus();

await QRX.storage.set("settings", { quality: "high" });
const settings = await QRX.storage.get("settings");

await QRX.wallet.requestPayment({
  recipient: "qrx...",
  amount: "0.1",
  memo: "Example purchase"
});
```

Mini SDK permissions

Permission	SDK capability	Security meaning
wallet.identity.read	QRX.wallet.getIdentity()	public wallet identity only
wallet.balance.read	QRX.wallet.getBalance()	balance read only
wallet.payment.request	QRX.wallet.requestPayment()	opens/pre-fills the normal Send flow; never signs
chain.read	QRX.chain.getHeight() and allowed chain reads	read-only local-chain information
network.status.read	QRX.network.getStatus()	read-only local network/node status
app.storage	QRX.storage.get/set()	app-ID-scoped local storage, capped at 64 KiB in v1

`requestPayment()` is intentionally a **request**, not a payment primitive. The app can suggest recipient, amount and memo, but the QRX wallet shows the normal Send screen and the user must review and confirm. The third-party app never receives the private key and cannot silently call a signing/broadcast path.

6.5 Installing and removing an app

Normal local workflow:

1. Open **Wallet** → **Apps**.
2. Select **Install .qrxapp**.
3. Choose the package.
4. Review name, author, requested permissions and unverified/sideload status.
5. Install.
6. Launch the new entry from the App Registry.
7. Remove it from the Apps management controls when no longer wanted.

A third-party app should be treated like locally installed software: install packages only from a source you intended to trust, even though the sandbox reduces the consequences of a malicious package.

6.6 Developer Mode and Hello QRX

Developer Mode loads an unpacked application directory live instead of making the developer rebuild a `.qrxapp` after every edit. It **does not disable the sandbox or permission checks**.

The reference SDK tree is:

```
GUIWALLET/qrxapp-sdk/  
├─ README.md  
├─ build-qrxapp.sh  
├─ demo-hello/  
└─ dist/hello-qrx.qrxapp
```

Build the demo package with:

```
cd GUIWALLET/qrxapp-sdk
./build-qrxapp.sh
```

The demo shows public identity/balance reads, chain/network reads, app-scoped storage and a safe payment request. Developers should copy the demo rather than reaching into undocumented Tauri internals.

6.7 0.0.10 App Platform ecosystem

0.0.9 is intentionally the compatibility base. The next layer is:



0.0.10 is where QRX adds developer identity/signature verification, immutable package publication through QRX Drive, QRX-Net discovery, the public/decentralized App Directory, update metadata/reputation and explicit Local/QRX Compute capability grants. The 0.0.9 manifest/SDK contract is the compatibility base so apps should not need a rewrite just to gain network distribution.

6.8 Localization and full-width Apps UX

The 0.0.9 Apps foundation is integrated with the same **55-locale wallet catalog** used by the rest of the GUI. App installation, App Registry state, permission review, Developer Mode, App Host labels, payment-request review and QRX Upscaler status/fallback text use translation keys rather than a separate English-only interface. Arabic also switches the document direction to RTL in the App Host. Technical identifiers such as `.qrxapp`, `QRX Compute`, `COMPUTE_POUC_V1`, `Mini JS SDK`, `Apple Silicon`, `Metal`, `Vulkan` and filter/runtime names remain invariant by design.

All 55 catalogs have the same **885-key** schema in this snapshot. The Phase 190 test verifies that the new Apps/Upscaler/App Host keys exist and are non-empty in every locale, that the localized UX is not merely an English fallback corpus, and that dynamic application paths call the active locale catalog. The normal i18n parity, editorial-quality and mixed-English lexical gates also pass.

The top-level **Apps** and **QRX Upscaler** views use the complete available wallet content width next to the persistent sidebar. Header, launcher, notices, Developer Mode and action/progress areas span the full workspace. The Upscaler may use two internal columns for local/compute information on wide displays and collapses to one column below 900 px. Dialogs, confirmation prompts and other narrow-reading surfaces intentionally remain bounded instead of being stretched across an ultrawide display. A launched third-party app receives the full App Host frame (`width:100%; height:100%`) while remaining inside its sandbox.

6.9 QRX Upscaler

QRX Upscaler is the first substantial Apps-area workload intended to exercise the same local-app principles. The current 0.0.9 source ships the deterministic local image/batch core, video-frame orchestration, hardware capability probing and the wallet entry.

Local examples:

```
qrx-upscaler image in.png out.png --scale 4 --filter lanczos3
qrx-upscaler batch frames-in frames-out --scale 2
qrx-upscaler video in.mp4 out.mp4 --scale 2
qrx-upscaler capabilities
qrx-upscaler selftest
```

Classical filters are `nearest`, `bilinear`, `bicubic` and `lanczos3` (default); scale is an integer factor from 1 to 8. PNG is available when built with libpng; PPM/PGM are always available.

The pixel accumulation path is fixed-point and has deterministic regression vectors. Do **not** use byte-for-byte equality of arbitrary GPU AI output as the future distributed-verification rule: heterogeneous Vulkan/Metal/CUDA inference can differ numerically. Distributed AI work must bind model/runtime/input/tile parameters and use the PoUC verification/challenge policy rather than assuming every GPU emits identical bytes.

Hardware capability profiles

The capability probe deliberately separates a **compatibility profile** from descriptive hardware metadata:

Device	QRX profile	Accelerator path	Default handling
Apple Silicon M1/ M2/M3/M4 and future compatible generations	apple-silicon	native Metal for QRX Metal backends; ncnn Vulkan path through MoltenVK	supported; RAM/ capability based Auto tuning
Raspberry Pi 5	raspberry-pi-5	VideoCore VII / V3DV Vulkan	supported low - memory path
ODROID - N2/N2+	odroid-n2-plus	Mali - G52 / PanVK	experimental; conservative tiles and classical fallback
other Vulkan - capable desktop	generic	native Vulkan	enabled only after loader + usable device/ runtime checks
no usable AI runtime	appropriate profile + unavailable runtime	CPU/classical	classical fallback remains available

Apple Silicon is one profile. M1 , M2 Pro , M3 Max , M4 , etc. are retained only as `generation / variant` metadata for display and scheduler hints. A future Apple Silicon generation can therefore enter the same `apple-silicon` compatibility path without inventing another platform profile.

On macOS, Vulkan is not treated as the native graphics API. The ncnn Vulkan backend uses **MoltenVK → Metal** when that runtime is present. The capability command reports this explicitly through `vulkan_mode=moltenvk` . Missing MoltenVK does not make the Mac unusable; it makes the AI Vulkan path unavailable while classical/local processing remains available.

Video and batch workflow

The intended video flow is:

```
video
-> inspect timing/audio
-> decode frames
-> process frame batches
-> restore frame order/timing
-> encode/remux
-> preserve audio when supported by the configured media tool
```

The current source executes the external media tool as a **separate process with an argument vector**, not through a shell command string and not linked into QRX Core. The release policy in this snapshot does not bundle FFmpeg; the operator can point `QRX_UPSCALER_MEDIA_TOOL` at an installed media tool. A self-contained media runtime can be packaged later only under an explicitly approved redistribution/license profile.

AI model/runtime status

The current source prepares capability-aware AI selection but does **not** yet bundle model weights. Candidate permissive runtimes include ncnn (BSD-3-Clause) and ONNX Runtime (MIT); each model's weights have their own license/provenance and must pass the normal QRX model governance rules. The preferred offline-photo direction discussed for the app is a Real-ESRGAN-class model through a permissive runtime, but the handbook distinguishes that planned runtime delivery from what the present source archive actually contains.

Distributed mode

Distributed frame batches are Proof-of-Useful-Compute work. They remain fail-closed until `COMPUTE_POUC_V1` reaches a governance-committed activation height (target not before 31 Jan 2027). A future distributed job uses bounded frame batches rather than one network job per frame, can reorder completed batches before assembly, and is paid from the user's compute escrow; it does not mint additional QUB. Private-video frames leaving the machine must remain an explicit per-job opt-in.

7. QRX Drive

Purpose: decentralized file storage and retrieval.

QRX Drive splits data into content-addressed pieces/shards, distributes them to providers, tracks storage contracts and can reconstruct files using multiple providers.

Upload workflow

1. Choose a file.
2. Select the intended storage profile (STANDARD , FAST , ARCHIVE where exposed).
3. Prepare the upload.
4. Review the contract/cost/placement information.
5. Start/advance the upload.
6. Wait until sufficient shards/providers are active.

Download workflow

QRX can use multiple providers in parallel. The runtime supports resume/retry and repair instead of forcing the whole file to restart after one failed shard.

Health and routes

The GUI can show authoritative shard routes on the Resource Globe. Privacy-protected regions are intentionally not exposed as precise locations.

File-size policy

QRX Drive is designed around chunking/large-object handling rather than an arbitrary small user-facing file cap. Practical limits still come from provider capacity, erasure layout, network policy, disk space and implementation limits.

Security

Private Drive content uses the QRX Drive encryption/key-envelope path.
Keep the wallet/recovery material safe: decentralizing ciphertext does not
make lost decryption keys recoverable by the network.

8. QRX-Net

Purpose: decentralized naming and web publishing.

QRX-Net adds `.qrx` domains and QRX-hosted site packages. The browser layer can resolve normal WWW content and QRX-native content without pretending `.qrx` is conventional DNS.

Domain workflow

Typical operations include:

- preflight a `.qrx` name,
- register,
- renew,
- update target/web root/publishing commitment,
- transfer,
- inspect history.

Site publishing

A site directory is packaged, content-addressed and prepared for QRX Drive distribution. Published versions can be listed and a previous version can be selected for rollback.

Browser security

The QRX browser sandbox accepts QRX-native names/paths under the QRX resolution rules. DApp bridge permissions are separate from generic page rendering.

Ads and youth safety

QRX-Net contains explicit policy layers for advertising/reward logic and youth-safety controls. These are not reasons to let arbitrary site code access the wallet without consent.

9. Resource Globe

Purpose: privacy-safe visual overview of QRX network resources.

The Globe aggregates storage, compute, model-cache and related provider information into coarse regions. It is intentionally not a public map of exact home addresses or private IPs.

AURA Compute · Automatic

For beginners this is the main AURA provider control.

Normal controls are intentionally small:

- **Off**
- **Automatic (recommended)**
- **Eco / Balanced / Performance**
- optional model-cache/disk budget

In Automatic mode QRX decides which compatible runtime and model/expert data to use.

What Automatic does

It can perform the chain:

```
hardware discovery
→ signed runtime selection
→ runtime verification/install
→ signed model catalog lookup
→ compatible model/expert placement
→ provider discovery
→ cache/prefetch
→ secure job execution
```

A normal user should not have to understand CUDA compute capability, GGUF quantization or MLX package paths.

Advanced provider controls

Advanced settings can expose runtime overrides, relay endpoints and operational settings. Change them only when diagnosing or intentionally pinning infrastructure.

10. Send

Purpose: send QUB or the supported quick-send assets from the active wallet.

Before confirming:

1. verify network,
2. verify destination chain/address,
3. verify amount and fee,
4. confirm the source wallet/address,
5. read any warning shown by the GUI.

QRX contains protections against common self-address and wrong-network mistakes, but no UI can make a correctly signed transfer to the wrong recipient reversible.

Use Address Book entries to reduce repeated copy/paste, but verify a new contact once before relying on it.

11. Receive

Purpose: show QRX/BTC receiving addresses and QR codes.

For QUB, the wallet may expose the primary/shared address and rotation features. Rotating/stealth-style privacy features should be used only when you understand how recovery is preserved.

For exchange deposits, use the address type explicitly supported by that exchange. Do not send centralized-exchange deposits to a stealth/shielded address merely because the wallet can generate one.

12. Transactions

Purpose: transaction history, status and accounting/export workflow.

Use this page to inspect incoming/outgoing activity and confirmations. The broader wallet stack supports exports by periods such as all-time, year, quarter and custom ranges so accounting does not require iterating an unbounded history in one request.

A transaction shown locally is not automatically final; check network/confirmation/finality state when that matters.

13. Staking

Purpose: stake QUB, delegate to validators and manage delegation lifecycle.

Core operations include:

- stake,
- delegate,
- undelegate,
- claim undelegated stake,
- inspect staking status and validator set.

Risks

Staking is not the same as a guaranteed bank yield. Validator behavior and protocol rules can affect rewards and penalties. Slashing/double-sign protection is consensus-critical.

14. Validator Mode

Purpose: operate the wallet/node as a validator.

A validator should be treated as always-on infrastructure, not as a casual checkbox.

Safe Pause

Use **Safe Pause** before a planned extended shutdown. The intent is to tell the validator lifecycle that the operator is going offline safely rather than abruptly disappearing while still expected to participate.

Validator fleet

The stack supports multiple validator signer identities/fleet management. Do not run the same validator key in two independent active processes that can double-sign.

Home validator catch-up

After downtime, let the node catch up and reach healthy chain state before expecting normal validator participation.

15. BTC Light

Purpose: lightweight Bitcoin wallet/service integration used by the GUI and cross-chain features.

The current architecture uses the shared Rust BTC wallet service and light synchronization rather than embedding a full Bitcoin Core node into QRX.

Keep separate concepts clear

- QUB wallet keys are not Bitcoin keys.
- QRX network sync is not Bitcoin sync.
- A BTC address must be backed up according to the BTC service/wallet design.

If BTC status looks stale, diagnose the BTC light service separately from QRX peer health.

16. Quantum Swaps

Purpose: cross-chain QUB/BTC exchange using HTLC/Velocity/SPV foundations.

High-level lifecycle:

```
order / session
→ hashlock and refund conditions
→ BTC funding
→ SPV verification / confirmations
→ QUB-side settlement
→ redeem OR timeout/refund
```

Funding transaction IDs and deadlines are security-sensitive. Do not casually replace a funding transaction after a swap has bound itself to a specific transaction/session rule.

Reorg handling and Bitcoin retarget/header validation belong to the Bitcoin-SPV security path, not to a GUI timer alone.

17. Markets

Purpose: QRX-native order book, chart and order control.

The chart uses QRX-native candles, trades and order-book state; it is not an embedded TradingView exchange widget.

LIMIT order workflow

1. select market,
2. choose BUY/SELL,
3. enter quantity and limit price,
4. set expiry when required,
5. review agent/owner identity,
6. acknowledge that the order is real on the selected network,
7. sign/broadcast.

The Markets view also includes cancel/order-control functions.

Do not confuse QRX-native market liquidity with an external exchange balance.

18. Agents & Kraken

Purpose: delegated trading/automation identities and external exchange gateway workflows.

Agent permissions are explicit. Typical boundaries include market allowlists, maximum trade values, daily limits, expiry and specific permission classes.

Paper vs live

A paper-trading or opportunity-analysis result is not a live external order. Live execution must be opt-in and use an authorized agent/gateway path.

Kraken

External Kraken actions require the separately configured credentials/permissions and sufficient exchange balances. Never embed a powerful unrestricted exchange API key into a public model prompt or QRX-Net page.

19. Privacy

Purpose: optional privacy features beyond normal transparent QUB.

The stack contains privacy layers for concepts including:

- receive-address rotation,
- stealth receiving,
- shielded QUB flows,
- hidden-balance/proof foundations,
- shield / private transfer / unshield lifecycle.

Important limitation

“Private” does not mean that every network fact disappears. Timing, peer/network observations, entry/exit transactions and other metadata can remain observable depending on the operation.

Transparent QUB remains the compatibility-first path for exchanges and basic transfers.

20. Safety Center

Purpose: beginner protection, backups and risky-operation separation.

Use the Safety Center before upgrading, migrating, enabling advanced privacy features or operating a validator.

Recommended routine:

- run safety check,
- create encrypted full-wallet backup,
- maintain a recovery phrase + matching `.qrseed` pair,
- verify backup health,
- keep at least one offline copy,
- test recovery before treating the backup as trustworthy.

Expert plaintext/private-key-style operations are intentionally not the default.

21. Roadmap

Purpose: show public development direction and feature status.

A roadmap item is not the same as a Mainnet guarantee. Distinguish:

- already implemented/tested,
- prepared/feature-gated,
- future target.

QRX Upscaler now has a shipped local foundation: deterministic classical image/batch processing, video frame orchestration, host capability detection and an Apps entry. AI model/runtime execution is a separate runtime layer and must be validated locally before use. Distributed provider execution remains gated behind `COMPUTE_POUC_V1` like every other post-Genesis resource layer.

For 0.0.9, the final documentation gate is this A–Z handbook plus the documentation coverage test.

22. Wallets

Purpose: manage multiple wallet names, inspect shared/legacy locations, import safely and switch active wallet.

The wallet manager is designed to avoid accidental overwrites. When importing an older wallet:

1. keep the source,
2. copy/import into the shared store,
3. inspect the identity,
4. activate it,
5. verify balance/history,
6. only later archive the old copy.

Never merge unrelated private key sets into one wallet directory.

23. Address Book

Purpose: local QUB/BTC contacts.

Contacts contain public information only: label, chain, address, tags and note. They must never contain private keys or recovery phrases.

The GUI keeps QUB and BTC entries explicitly separated. Export/import uses JSON and is designed not to silently overwrite pinned contacts.

24. AURA — beginner guide

24.1 What AURA is

AURA is the user-facing distributed useful-compute/AI layer on QRX. The network can combine many heterogeneous providers rather than requiring every user to own a datacenter GPU.

24.2 Automatic is the default

The intended beginner choice is:

```
AURA Compute: Automatic
Power profile: Balanced
Disk cache: Automatic or a simple GiB limit
```

That is enough.

24.3 Hardware auto-detection

QRX can discover:

- native CPU resources,
- Apple Silicon/Metal capability,
- NVIDIA CUDA through the driver API when present,
- memory/capability information used by the MoE scheduler.

NVIDIA P40/Pascal is explicitly representable and is not incorrectly assumed to have tensor cores.

24.4 Signed runtime packages

0.0.9.45 adds a signed runtime-package catalog. A runtime announcement binds:

- publisher identity,
- package ID/version,

- platform/architecture,
- runtime/backend type,
- hardware requirements,
- content hash,
- download URI,
- validity heights and anti-rollback sequence.

The One-Click activation flow is:

```
native hardware discovery
→ choose compatible signed runtime
→ fetch package
→ SHA3 verify package
→ atomically install
→ activate adapter
→ persist host config
```

A modified package is rejected before activation.

24.5 Model catalog and discovery

Models are not identified by random filenames. A signed QRX model catalog identifies model/version, architecture, format, quantization, license, context, capability and a **manifest root**.

The manifest lists content-addressed assets such as:

- config,
- tokenizer,
- shared weights,
- weight chunks,
- MoE expert packs,
- adapters.

Provider availability gossip then tells the network which provider currently holds which assets.

24.6 Why QRX does not ship every model inside the wallet installer

Large model weights can be many gigabytes. Shipping all supported models with every wallet would make installation impractical and would duplicate data unnecessarily.

Instead the normal hierarchy is:

```
local RAM / VRAM
→ local SSD model cache
→ LAN peer
→ current MoE pod
→ nearby QRX provider
→ QRX Drive
→ external origin fallback
```

Popular model assets should therefore become increasingly QRX-native over time.

24.7 External origins, Hugging Face and initial model seeding

QRX 0.0.9 Genesis Freeze adds a real origin/bootstrap path rather than treating an external URL as a placeholder. The shipped data-only catalog is:

```
config/aura/models/official-origin-catalog.qrx
```

It contains zero-touch Nano/Edge GGUF bootstrap specifications for Qwen 0.6B/1.7B/4B, DeepSeek R1 Distill Qwen 1.5B and Ministral 3B, plus larger Qwen3.8-27B, Kimi-K2-Instruct, Kimi-K3 and DeepSeek-V3.2 entries. A catalog specification is **not yet trusted model bytes**. Before an official model can enter the governed QRX model catalog, a seed operator imports the selected upstream revision once. QRX then:

```
resolve upstream repository
-> pin immutable revision SHA
-> verify declared license/gating policy
-> enumerate selected runtime/model files
-> download with resume support
-> hash every object into QRX SHA3 CAS
-> parse config + Safetensors shard index
-> derive MoE layer/expert ranges where available
-> build source map + QRX model bundle + model registry record
-> publish/sign the resulting manifest through QRX model
governance
-> announce availability
-> replication controller places missing copies
```

A normal provider does not need to know Hugging Face. Its automatic fetch order remains QRX-first. If the governed model manifest permits an external fallback and QRX Drive/CAS cannot supply an object, the client may fetch only the **exact immutable revision** recorded by that manifest. A server returning different bytes or a different revision is rejected by content-root/revision checks.

24.8 Nano/Edge zero-touch models and graceful scaling

AURA Automatic is designed to remain useful even when the entire available compute fabric is one small machine. The official zero-touch catalog therefore contains execution-ready, non-gated GGUF choices in several capacity bands rather than starting with giant MoE models:

- `qrx/qwen3-0.6b-q8` — Nano fallback for very small hosts;
- `qrx/deepseek-r1-distill-qwen-1.5b-q4` — small reasoning-oriented Nano model;
- `qrx/qwen3-1.7b-q8` — stronger general/coding Nano model;
- `qrx/ministral3-3b-instruct-q4` — Edge model with strong tool-use weighting;
- `qrx/qwen3-4b-q4` — Edge general/coding model.

The user does **not** choose these names. AUTO combines measured free memory, measured throughput, task class and task complexity. A low-memory host can answer locally with Nano; an 8 GiB Pi-class host can move between Nano and Edge; larger hosts or multiple providers can

escalate to Local/Cluster/K2/K3 classes. If a host cannot execute an LLM efficiently it can still contribute routing, verification, model cache, relay, pre/post processing or QRX Drive capacity.

For Apple Silicon GGUF models, native Metal discovery selects the `LLAMA_CPP_METAL` adapter. This is intentionally separate from MLX: Metal hardware support does not falsely claim that an MLX runtime is installed. CPU, CUDA and Metal runtime choice remains hidden under AUTO.

Meta Llama is **not required** for the default zero-touch set. Official Llama access uses Meta's community license and may require explicit access/license acceptance. QRX therefore keeps Llama entries in `optional-gated-origin-catalog.qrx`, with `auto_default=0` and `gated_optional=1`. An Advanced user/operator can enable such a source deliberately; normal Automatic never blocks first run on a gated model.

The model weights are not bundled in the QRX application archive. The first authorized import downloads the exact governed quantization, pins the immutable upstream revision, verifies/content-addresses it, seeds QRX Drive/CAS and allows provider replication. Subsequent clients prefer QRX peers/Drive before the external origin.

Operator tool:

```
# Inspect upstream metadata without downloading the model
qrx-aura-model-seed \
  --catalog config/aura/models/official-origin-catalog.qrx \
  --model qrx/qwen3.8-27b \
  --resolve-only

# Perform the one-time content import/QRX-CAS seed
qrx-aura-model-seed \
  --catalog config/aura/models/official-origin-catalog.qrx \
  --model qrx/qwen3.8-27b \
  --cas /srv/qrx/model-cas \
  --work /srv/qrx/model-import
```

For gated repositories, a token can be supplied only through an environment variable using `--hf-token-env`; it is never written into the model manifest. Gated imports stay disabled unless explicitly allowed.

Very large models can require hundreds of GB to more than a TB of upstream data. The presence of a bootstrap catalog entry therefore does **not** mean the source archive itself contains those weights. The model becomes QRX-native after the initial seed has produced real content roots and the governed manifest is published.

24.8 Model replication

The replication controller can consider:

- current replica count,
- region diversity,
- demand,
- catalog priority/popularity,
- boot-critical assets,
- hot MoE experts.

Under-replicated assets receive higher placement priority. Hot expert packs can be migrated toward demand.

24.9 Governance, provenance and licenses

0.0.9.46 adds a separate governance boundary for model updates.

A model can be rejected because:

- its license is not permitted,
- required provenance is missing/mismatched,
- no effective governance decision exists,
- the manifest is blocked,
- a rollback requires a different previous manifest.

A governance approval cannot override an explicit license deny rule.

24.10 NAT and relays

A home provider does not have to expose a public inbound port just to participate. The 0.0.9.41 relay design supports an **outbound provider tunnel**.

The relay transports framed data; signing, PQ key establishment and authenticated encryption remain endpoint responsibilities between requester and provider.

24.11 Durable jobs and retries

Jobs/results are journaled so a crash or lost response does not automatically mean the same useful computation has to be paid/executed again.

An **identical retry** can return the committed result. A modified request that reuses the same sequence is rejected.

24.12 Eco / Balanced / Performance

These profiles affect scheduling preference, not model truth.

- **Eco** — stronger preference for lower cost/energy.
- **Balanced** — mixed quality/latency/cost/energy.
- **Performance** — stronger preference for quality, latency and throughput.

0.0.9.47 routes production offers using quality, latency, price, energy, reliability and locality rather than “fastest GPU wins”.

25. AURA — advanced/operator reference

25.1 Host contract

The GUI writes a durable AURA host configuration using the `QRXAURA41` format. The node receives its path through `QRX_AURA_PROVIDER_CONFIG`.

Important host fields include:

- provider and pod identity,
- network/region,
- cache/memory/network limits,
- secure-dispatch and PQ requirements,
- runtime adapter selection/path,
- lease and job journals,
- optional relay endpoints.

25.2 Runtime adapters

Stable adapter kinds are:

- `AUTO`
- `LLAMA_CPP_CPU`
- `LLAMA_CPP_CUDA`
- `MLX_METAL`
- `EXTERNAL`

Beginners should leave this on `AUTO`.

25.3 MoE pods

A pod can advertise roles such as inference, router, embedding, RAG, pre/post-processing, verification and model cache. Scheduling is benchmark/capability driven rather than tied to a specific consumer hardware brand.

25.4 Expert locality and predictive prefetch

The coordinator tracks co-activation and can prefetch experts likely to be needed next. This reduces WAN traffic and activation latency when expert sequences are predictable.

25.5 Supervisor and relay failover

The provider supervisor records runtime health and relay failures. Relays can enter cooldown and a healthier relay can be selected automatically.

25.6 Production telemetry

Production telemetry can track requests, tokens, latency, observed quality, cost, energy, WAN bytes and failures. Telemetry commitments make benchmark/campaign outputs reproducible without pretending that a single benchmark predicts every future task.

26. Native assets and tokenization

QRX native assets support issuance concepts plus regulated-token controls such as whitelist/revoke policy where the asset definition enables them. Treat issuer authority as a security and legal property of that asset; it is not equivalent to QUB itself.

Asset anti-spam economics and burn/fee accounting exist to stop unlimited free namespace/state abuse.

27. CLI and RPC quick reference

The CLI accepts:

```
qrx-cli [--network <alpha|testnet|regtest|mainnet>]
        [--datadir PATH]
        [--wallet NAME]
        [--rpc-user USER]
        [--rpc-password PASS]
        <command>
```

27.1 Node/status

```
qrx-cli getinfo
qrx-cli getblockcount
qrx-cli getblockchaininfo
qrx-cli getnetworkinfo
qrx-cli getnodestatus
qrx-cli getmainnethealth
qrx-cli getpeerinfo
qrx-cli getmempoolinfo
qrx-cli getbuildinfo
```

27.2 Wallet

```
qrx-cli getwalletinfo
qrx-cli getnewaddress
qrx-cli listaddresses
qrx-cli getbalance
qrx-cli history
qrx-cli sendtoaddress <addr> <amount> [memo]
```

27.3 Staking/validator

```
qrx-cli getstakinginfo
qrx-cli stake <amount>
qrx-cli delegate <validator> <amount>
qrx-cli undelegate <validator> <amount>
qrx-cli claim-undelegated <validator>
qrx-cli validator-set
qrx-cli getvalidatorstatus
```

27.4 QRX Drive

```
qrx-cli listdrivefiles [owner]
qrx-cli getdrivefilehealth <contract_id>
qrx-cli getdriveshardroutes <contract_id>
qrx-cli preparedriveupload <source> <STANDARD|FAST|ARCHIVE>
qrx-cli startpreparedriveupload <contract_id> <prepare_id>
qrx-cli startdrivedownload <contract_id> <destination>
qrx-cli listdrivetransfers
```

27.5 QRX-Net

```
qrx-cli getdomainpreflight <name.qrx> [years]
qrx-cli registerdomain <name.qrx> <years> [qub_address]
qrx-cli renewdomain <name.qrx> <years>
qrx-cli listdomains [owner]
qrx-cli prepareqrxsite <name.qrx> <folder>
qrx-cli listqrxsiteversions <name.qrx>
qrx-cli rollbackqrxsite <name.qrx> <version>
qrx-cli fetchqrxsite <name.qrx> [path]
```

27.6 AURA/resource

```
qrx-cli getresourcedashboard  
qrx-cli getresourceatlas  
qrx-cli getaurafabric  
qrx-cli getauraatlas  
qrx-cli listauramodels  
qrx-cli gethostingmissions
```

27.7 Privacy

```
qrx-cli privacy-feature-status  
qrx-cli stealth-address  
qrx-cli stealth-scan  
qrx-cli shielded-address  
qrx-cli shielded-balance
```

The CLI source is the authoritative command inventory for the exact build; use `qrx-cli --help /usage` output when scripting against a different snapshot.

27A. Genesis memo, asset burns and staged post-Genesis activation

The canonical Mainnet Genesis memo is part of `genesis.cfg` and therefore part of the Genesis hash:

At the threshold of the AI age, amid global change, QRX was launched to keep value, verification, and digital sovereignty in the hands of people – and to help ensure that the opportunities of artificial intelligence are open to all, not reserved for a few.

Changing even one character before finalization changes the Genesis hash. After the final Genesis is distributed, the memo is immutable as part of chain identity.

Native-asset anti-spam burns use `block_reward_units_v1`. Genesis prices preserve the historical amounts (for example MAIN = 400 current block rewards = 100 QUB at the 0.25-QUB subsidy), then scale with the subsidy after halvings instead of becoming a fixed-supply bottleneck. If subsidy eventually reaches zero, the consensus helper retains a one-atom-per-unit anti-spam floor.

Post-Genesis resource features are fail-closed and activate by threshold-signed **height**, never merely because a calendar date arrives. Operational target dates are:

- `DRIVE_V1` : 2026-11-30 17:00 UTC
- `QRX_NET_V1` : 2026-12-07 17:00 UTC
- `ADVERTISING_V1` : 2026-12-15 17:00 UTC
- `COMPUTE_POUC_V1` : 2027-01-31 17:00 UTC

`ADVERTISING_V1` additionally requires `QRX_NET_V1`, so domain/hosting activation no longer automatically enables the advertising economy. Governance should schedule the final activation heights only after the relevant readiness gates and soak tests pass.

27A.1 Common protocol-readiness state machine

All four staged post-Genesis layers use one common status model rather than four unrelated wallet rules:

LOCKED

- > NOT_READY
- > SOAKING
- > WAITING_TARGET_DATE
- > READY_FOR_GOVERNANCE
- > SCHEDULED
- > ACTIVE

Soak means an observation/burn-in period on the real Mainnet state. Reaching a provider count for a moment is not enough; the prerequisite layer must remain stable for the configured number of blocks. Target dates are **not-before policy dates**, not automatic consensus switches. A feature becomes scheduled only after a valid threshold-signed on-chain governance transaction commits its activation height.

Mainnet does **not** trust a local `protocol_upgrades.db` file as activation authority. Protocol schedules are consensus state; all validating nodes derive the same activation height from the chain.

The current staged policy is:

Layer	Target date	Automatic readiness / soak policy
DRIVE_V1	30 Nov 2026	>=14 serving and mature attested providers, >=14 independent operators, >=4 ASNs, >=3 visible regions, >=14 GiB proven capacity, >=80% availability, >=80% proof success, health >=60, ~7-day provider maturity soak
QRX_NET_V1	07 Dec 2026	DRIVE_V1 active, >=14 serving providers, >=4 ASNs, >=3 regions, >=3 active storage contracts, >=85% availability, health >=70, ~7 days of active Drive soak
ADVERTISING_V1	15 Dec 2026	QRX_NET_V1 active, >=14 serving providers, >=4 ASNs, >=3 regions, >=3 active .qrx domains, >=85% availability, health >=70, ~7 days of active QRX-Net soak
COMPUTE_POUC_V1	31 Jan 2027	DRIVE_V1 + QRX_NET_V1 active, >=8 bound compute-provider identities, all nine PoUC safety/readiness evidence flags, >=14 storage providers, >=4 ASNs, >=3 regions, health >=70, ~14-day dependency/shadow soak

Before `DRIVE_V1` , storage providers may perform only the non-economic preflight operations required to prove capacity, bind discovery identity, activate and collect attestations/maturity. Storage contracts/rewards remain gated. Before `COMPUTE_POUC_V1` , a provider may bind its cryptographic compute-provider identity so Mainnet can count real prospective providers; paid compute escrows/jobs/rewards remain gated.

The wallet reads this state from its local `qrx` via:

```
getprotocolreadiness DRIVE_V1
getprotocolreadiness QRX_NET_V1
getprotocolreadiness ADVERTISING_V1
getprotocolreadiness COMPUTE_POUC_V1
```

The GUI may therefore show, for example, `NOT READY - 3/14 providers` , `SOAKING - 5/7 days` , `LOCKED - DRIVE_V1 dependency` , or `READY FOR GOVERNANCE` . `READY FOR GOVERNANCE` means the machine-verifiable network criteria pass; it does not replace human review of release/security findings before the governance signers approve an activation proposal.

27A.2 Three-Mac governance activation example

Three separate governance signers are sufficient for the existing 3-of-5 threshold. They are **not** sufficient to fake readiness. For example, three Macs can hold `GOV1/GOV2/GOV3` and sign a `DRIVE_V1` proposal, while the readiness engine may still report only `3/14` independent storage providers. Once readiness passes, the three signers approve the same proposal, one node broadcasts the `GOVERNANCE_PROTOCOL` transaction, and every validating node learns the same future activation height from consensus state. No per-validator file copying is required.

28. Security model

28.1 Wallet keys

Protect the hybrid private keys and recovery material. QRX can verify network data; it cannot undo disclosure of a private key.

28.2 PQ boundaries

QRX uses hybrid/post-quantum components in selected protocol paths, including wallet signatures and AURA/Drive security work. "PQ-capable" should not be interpreted as proof that every dependency, platform or cryptographic path has undergone independent post-quantum audit.

28.3 Model/runtime trust

AURA separates:

- runtime package publisher signature,
- package content hash,
- model catalog signature,
- model manifest root,
- provenance,
- license policy,
- governance decision,
- provider availability signature.

This is intentionally more strict than "download the newest file named model.gguf".

28.4 Relays are not trusted with endpoint keys

A relay should not need the provider wallet private key or the requester session secret. Treat a relay as transport infrastructure, not as an execution authority.

28.5 Fail closed for production

0.0.9.48 release readiness requires secure host settings, a compatible verified runtime package path, governed recommended models and a distributable model path before reporting AURA release readiness.

28.6 Governance Vault and 3-of-5 operator safety

QRX Mainnet governance remains **3-of-5** at consensus level. The Governance Vault is an operator-safety layer and does not weaken or replace that threshold.

Recommended layout:

OFFLINE BACKUP VAULT

```
DEV_GOV_1 encrypted backup
DEV_GOV_2 encrypted backup
DEV_GOV_3 encrypted backup
DEV_GOV_4 encrypted backup
DEV_GOV_5 encrypted backup
signing disabled
```

OPERATIONAL VAULT

```
DEV_GOV_1 ONLINE private key
DEV_GOV_2 ONLINE private key
DEV_GOV_3 OFFLINE public descriptor only
DEV_GOV_4 OFFLINE public descriptor only
DEV_GOV_5 OFFLINE public descriptor only
```

An operational vault hard-limits itself to **two online private signing keys**. The remaining roots are descriptor-only. A third signature is produced on a separate/offline signer and imported as a normal immutable governance signature file. The existing `governance-apply` path still verifies three unique valid Genesis governance roots before applying a proposal.

The offline five-key backup stores encrypted private-key material under the non-signing filename `governance.key.backup`; the vault API refuses to sign from an `OFFLINE_BACKUP` vault. Restoring a root into an operational vault is explicit and still cannot raise the online signer count above two.

Useful commands:

```
governance-vault-backup-create <backup> <GOV1> <GOV2> <GOV3>
<GOV4> <GOV5>
governance-vault-init <operational-vault>
governance-vault-restore-online <backup> DEV_GOV_1 <operational-
vault>
governance-vault-restore-online <backup> DEV_GOV_2 <operational-
vault>
governance-vault-add-offline <operational-vault> <DEV_GOV_3/
governance.pub>
governance-vault-status <operational-vault>
governance-vault-sign <operational-vault> DEV_GOV_1 <proposal>
<signature>
```

The five-key backup is intended for **offline/removable encrypted storage**, not permanent attachment to an internet-connected wallet. Keeping all five recoverable in one backup location is convenient, but geographic duplication and distinct per-key passphrases are still strongly recommended.

29. Upgrades, rollback and migration

Before an upgrade:

1. stop initiating new risky operations,
2. verify node health,
3. create/verify wallet backup,
4. back up important operator configuration,
5. install the new version,
6. allow migrations to complete,
7. re-check node/network/wallet state,
8. only then resume validator/provider operation.

0.0.9.48 includes an idempotent AURA host migration helper that can initialize missing runtime-auto state, secure defaults, journals and a conservative cache budget.

Rollback boundaries

Code rollback and model rollback are separate.

AURA model rollback uses signed governance and a previously known manifest root; it is not a request to accept arbitrary older bytes.

Database/consensus rollback should use the dedicated recovery/undo paths rather than manually deleting random state files.

30. Troubleshooting decision tree

Daemon offline

1. Confirm selected network.
2. Confirm the expected data directory.
3. Start/restart the local node.
4. Check whether another process owns the listen/control port.
5. Check logs/build info.
6. Do not delete chain/wallet state as a first troubleshooting step.

Wallet will not unlock

1. Confirm the correct wallet name.
2. Confirm Caps Lock/input layout.
3. Try the expected legacy empty-passphrase behavior only when the wallet is known to be legacy.
4. Inspect wallet key protection in Wallets/Safety Center.
5. Use recovery into a new wallet instead of editing encrypted PEM files by hand.

No peers / not syncing

1. Check `getnetworkinfo`, `getpeerinfo`, `getnodestatus`.
2. Confirm firewall/network.
3. Confirm network profile.
4. Check seed/addnode configuration.
5. Compare height with another known-good node.

QRX Drive object degraded

1. Open file health.
2. Inspect shard routes/provider count.
3. Resume any interrupted transfer.
4. Let repair orchestration seek replacement providers.

5. Do not delete the only local recoverable copy until redundancy is healthy.

AURA Automatic says no compatible runtime

1. Keep Runtime on AUTO.
2. Confirm native hardware discovery.
3. Confirm the signed runtime catalog contains a package for platform/arch/backend.
4. Confirm package validity heights and publisher key.
5. Confirm the package hash matches after download.
6. For CUDA, verify the NVIDIA driver can expose the device.
7. For Apple Silicon, Metal discovery alone does not pretend MLX is installed; the verified MLX adapter/package completes that transition.

AURA provider not reachable behind NAT

1. Enable/configure a relay endpoint.
2. Confirm outbound connectivity.
3. Check supervisor relay health/cooldown.
4. Do not expose wallet private keys to the relay.

AURA retry fails

If a request is an exact retry after a lost result, the durable journal should allow returning the committed result. If the payload changed while reusing the same sequence, rejection is intentional anti-replay behavior.

Model exists but cannot be selected

Check:

- model catalog validity,
- governance status,
- license policy,
- provenance,
- hardware/runtime requirements,
- context requirement,
- model/expert availability,
- disk budget,

- current provider health.

BTC Light stale

Diagnose BTC service/sync separately from QRX peers. A healthy QRX node does not imply a healthy Bitcoin light sync.

31. Developer/integrator map

Important source areas:

qrx-core/src/compute/ layers	AURA, PoUC, model/runtime/provider
qrx-core/src/storage/	QRX Drive and storage transport/runtime
qrx-core/src/net/	QRX-Net names/sites/browser/policy
qrx-core/src/consensus/	finality/slashing/consensus pieces
qrx-core/src/bitcoin/	Bitcoin SPV
GUIWALLET/	Tauri desktop wallet
scripts/	release/build/audit tooling

0.0.9 closing modules

- `qrx_aura_runtime_packages.*` — signed runtime packages + one-click activation
- `qrx_aura_model_governance.*` — governance/provenance/license/update/rollback
- `qrx_aura_production_routing.*` — quality/cost/energy/WAN production routing and telemetry
- `qrx_aura_release.*` — host migration and release-readiness gate

Adding a runtime

Do not only drop a `.so/.dylib/.dll` into a folder. Add a signed package announcement with platform/arch/backend requirements and content root, then verify it through the runtime package manager.

Adding a model

A new model generation should normally require a new signed catalog/manifest entry, **not a QRX consensus fork**. Governance, license and provenance rules still apply.

Adding a QRX app in 0.0.9

Start from `GUIWALLET/qrxapp-sdk/demo-hello/` , request only the permissions the app needs, and test it first in Developer Mode. Package it with `build-qrxapp.sh` , then install the resulting `.qrxapp` through the wallet. Do not call undocumented Tauri commands from third-party code; the Mini SDK/App Host is the stable boundary.

The 0.0.9 SDK is intentionally small. A capability that is not in the allowlist is unavailable to third-party apps rather than implicitly granted.

32. FAQ

Do I have to download every AURA model?

No. Model/expert assets are fetched and cached according to hardware, demand, placement policy and disk budget.

Do I choose Qwen/Kimi/DeepSeek manually?

Normally no. Automatic routing should select a compatible model for the task. Advanced users can inspect/pin behavior when explicitly supported.

Does every AURA provider need a public IP?

No. NAT/relay mode is designed for providers that can make an outbound connection but cannot accept an inbound public port.

Can QRX use a MacBook with 8–16 GB?

Potentially yes, especially for smaller models, utility roles or selected MoE expert packs. Placement is capability/budget driven rather than “full model or nothing”.

Are P40 GPUs supported?

The runtime capability layer explicitly represents NVIDIA P40/Pascal SM 6.1 and avoids assuming tensor cores on that generation. An appropriate compatible CUDA runtime/model path is still required.

Is AURA a single model?

No. AURA is the orchestration/user layer over a signed, versioned model fabric.

Does QRX Drive have one central storage server?

No. The target architecture uses distributed providers, redundancy, multi-provider reconstruction and repair.

Can I recover without my wallet backup?

Do not assume so. Decentralization does not recreate lost private keys. Maintain verified recovery material.

Can third parties build apps for the QRX Wallet?

Yes. 0.0.9 provides the local `.qrxapp` format, App Registry, sandbox, permissions, Mini JS SDK and Developer Mode. Public/decentralized distribution is deferred to 0.0.10.

Can a `.qrxapp` spend my QUB by itself?

No. The v1 SDK can request that the wallet open/pre-fill a payment, but the user reviews and confirms the normal Send flow. The app does not receive private keys or a silent signing primitive.

Is Apple M1/M2/M3/M4 a different Upscaler platform profile?

No. They all use the single `apple-silicon` compatibility profile. Generation and chip variant are metadata used for display/automatic tuning; RAM and verified runtime capability remain more important than the marketing generation name.

Is Vulkan native on Apple Silicon macOS?

No. QRX treats Metal as the native Apple graphics API. An ncnn Vulkan backend can run through MoltenVK when that runtime is available.

Is a passed 123/123 test suite a security audit?

No. It is a regression signal for the shipped test matrix, not an independent security certification.

33. 0.0.9 release checklist

For the final 0.0.9 branch close:

- **DONE:** AURA host/provider integration
- **DONE:** Durable job/result retry journal
- **DONE:** NAT/relay transport
- **DONE:** Production runtime adapters
- **DONE:** Zero-config model fabric and provider discovery
- **DONE:** Autonomous replication repair/hot expert placement
- **DONE:** Provider supervisor/relay failover/health
- **DONE:** Signed runtime package manager + one-click activation
- **DONE:** Model governance/provenance/license/update/rollback
- **DONE:** Quality/cost/energy production routing + telemetry
- **DONE:** Mainnet AURA migration/readiness gate
- **DONE:** Real immutable-origin/Hugging-Face bootstrap + resumable import
- **DONE:** Qwen/Kimi/DeepSeek bootstrap catalog specifications
- **DONE:** Drive-first governed origin fallback and executable replication transfer
- **DONE:** Complete A-Z handbook source
- **DONE:** Governance Vault V1: offline five-key backup, max-two operational signers, offline signature import
- **DONE:** On-chain GOVERNANCE_PROTOCOL activation scheduling; local Mainnet upgrade files are non-authoritative
- **DONE:** Common DRIVE/QRX-Net/Advertising/Compute readiness framework and wallet roadmap status
- **DONE:** Final documentation coverage test
(compute_phase174_qrx_a_to_z_documentation_gate)
- **DONE:** QRX App Foundation v1: .qrxapp , App Registry, sandbox/App Host and per-call permissions
- **DONE:** QRX Mini JS SDK v1 + Hello QRX demo + Developer Mode
- **DONE:** QRX Upscaler local classical image/batch/video orchestration and capability probe
- **DONE:** Unified apple-silicon Upscaler compatibility profile with M-generation/variant metadata
- **DONE:** Phase 190 multilingual Apps/Upscaler/App Host UX across all 55 locale catalogs (885-key parity) + full-width Apps/Upscaler workspaces

- **DONE:** Raspberry Pi 5 supported and ODROID-N2/N2+ experimental Vulkan profiles with safe fallback
 - **OPEN:** Native AI upscaler runtime/model bundle on every release target (separate runtime-delivery gate)
 - **OPEN:** 0.0.10 developer signatures / QRX Drive / QRX-Net / App Directory / app compute capabilities
 - **OPEN:** Native Tauri builds on each release OS/architecture in CI
 - **OPEN:** Public-release code signing/notarization where applicable
-

34. Glossary

AURA — QRX useful-compute/AI orchestration layer.

Asset manifest — content-addressed description of model files/expert packs.

BFT — Byzantine fault tolerant consensus/finality family.

Content root — cryptographic hash identifying exact bytes/content.

Expert — MoE sub-network selected by a router for some tokens.

Hot expert — expert with elevated current demand.

Manifest root — cryptographic identity of a complete model manifest.

ML-DSA-65 — post-quantum digital-signature algorithm used in QRX hybrid wallet/signature work.

MLX/Metal — Apple Silicon-oriented model runtime/backend path.

MoE — Mixture of Experts model architecture.

PoSTOR / Proof of Storage — QRX storage proof/provider layer.

PoUC — Proof of Useful Compute.

Provider — node offering storage/compute/network resources.

QRX Drive — decentralized storage layer.

QRX-Net — decentralized naming/site/browser layer.

Relay — transport intermediary used when a provider cannot accept inbound connections.

Runtime adapter — executable bridge from QRX AURA jobs to an inference runtime.

SPV — Simplified Payment Verification for Bitcoin header/transaction proof validation.

QRX App Host — trusted wallet-side mediator between sandboxed `.qrxapp` code and permission-checked native APIs.

QRX Mini JS SDK — injected JavaScript API exposed to third-party QRX apps; v1 is read-oriented plus safe payment requests and app-scoped storage.

.qrxapp — bounded ZIP application package with `qrx-app.json` manifest used by the 0.0.9 local App Registry.

Apple Silicon profile — single QRX Upscaler compatibility profile for Apple ARM Macs; M-generation/variant is tuning metadata, not a separate platform.

MoltenVK — Vulkan-on-Metal translation layer used when QRX runs an

ncnn Vulkan path on macOS.

Velocity — QRX high-throughput/native market/cross-chain execution family.

Documentation authority

This Markdown file is the canonical 0.0.9 offline handbook. Release notes describe deltas; this handbook describes how the complete system is intended to be used.

When this guide disagrees with an exact command accepted by a newer binary, the newer binary's versioned CLI/API contract wins and the documentation coverage gate should be updated in the same release.

AURA zero-touch runtime delivery

For normal users, AURA runtime setup is automatic. The user does **not** choose CPU vs. CUDA vs. Metal, GGUF, quantization, or a llama.cpp build. AURA Compute -> Automatic is the normal path.

At startup, QRX discovers the host hardware, selects a compatible signed runtime package, verifies the package SHA3 content root and publisher signature, probes the QRX AURA plugin ABI, and only then persists the runtime as active. Failed verification leaves the package inactive.

The Genesis runtime matrix covers:

- Linux x86-64: llama.cpp CPU, plus CUDA when a compatible NVIDIA GPU is present.
- Linux ARM64: llama.cpp CPU/ARM path for Raspberry Pi 5 and other ARM64 systems.
- macOS x86-64: llama.cpp CPU.
- macOS Apple Silicon: llama.cpp Metal for GGUF models; MLX remains a separate verified runtime path for MLX-formatted models.
- Windows x86-64: llama.cpp CPU; CUDA may be supplied by an enabled native CUDA release runner.
- NVIDIA CUDA packages include compute capability 6.1 so Tesla P40-class providers remain eligible.

Runtime lookup is content-addressed. QRX prefers a verified local CAS object, may use an available QRX Drive/provider context, and only then falls back to the signed HTTPS release origin. A package downloaded from an origin is rejected if its bytes do not match the content root in the signed runtime catalog.

The runtime catalog is signed by the QRX runtime publisher key. The public trust root is bundled with the wallet/application resources; it is not trusted merely because a download server supplied a key next to a package. This prevents a compromised package host from replacing both a binary and the key used to approve it.

`qrx` consumes the wallet-generated AURA host configuration. When AURA is set to Automatic and no explicit runtime override is present, daemon startup performs the zero-touch runtime bootstrap. If the release catalog is unavailable, the node can still start while AURA remains pending; an unverified runtime is never silently activated.

Release-operator requirements

Zero-touch for the end user still requires the release operator to prepare the trusted artifacts before shipping a platform installer:

1. Build the native runtime artifacts on the appropriate native/CI runners.
2. Sign the unified runtime catalog with the offline/CI-protected Ed25519 runtime signing key.
3. Bundle the resulting public publisher key and signed catalog in the wallet resources.
4. Package/sign/notarize the platform application as required by the operating system.
5. Seed the verified runtime artifacts into the release origin and, where available, QRX Drive.

The private runtime signing key must never be included in the source tree or wallet.